

最优化技术

孙志强

第六章 启发式搜索方法求解全局优化问题

内容

1 6.1 概述

2 6.2 进化算法

3 总结

概述

前面系统的学习了线性规划和非线性规划的求解方法。对于非线性规划而言，诸如SQP、罚函数法等求解方法都是针对局部最优解的，即最优解只是在它的一个邻域内最小。只有当问题为凸规划时，局部最优解才变为全局最优解。但并不是所有的问题都是凸规划，在这种情况下如何获取问题的全局最优解？这就是全局优化所关心的问题。

全局优化研究的是非线性函数在某个区域上全局最优点的特征和计算方法，发端于20世纪60年代，至今仍处于不断发展的过程中。

示例

我们来看一个例子

$$(P) \begin{cases} f(x) = e^{-x^2} \sin(6x) \rightarrow \min \\ -2 \leq x \leq 2 \end{cases}$$

利用Matlab中的fmincon函数进行求解：

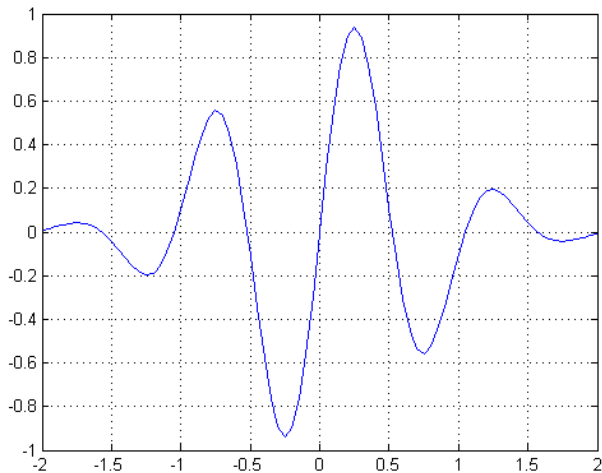
```
X = fmincon(@(x) exp(-x^2)*sin(6*x), 0.5, [], [], [], [], [], -2, 2)
```

初始点为0.5，最优解为-0.2481。将初始点修改为-0.5：

```
X = fmincon(@(x) exp(-x^2)*sin(6*x), -0.5, [], [], [], [], [], -2, 2)
```

最优解为0.7449。由此可见，非线性规划的局部最优解受到初始点的影响比较大。

示例



全局优化问题的算法及工具

算法

- 1 进化算法(Evolutionary algorithm)
- 2 分支定界法(Branch and bound method)
- 3 割平面法(Cutting plane method)
- 4

工具

- 1 Matlab global optimization toolbox
- 2 Mathematica
- 3 Lingo
- 4

进化算法主要特性

- 在自然选择和遗传过程的启发下产生的，具有自组织、自适应、自学习和并行性等优点；
- 擅长于处理组合优化、非凸光滑优化、多态优化和混合规划等问题；
- 对目标函数的要求不高，不容易陷入局部最优；
- 很多时候甚至不需要目标函数存在明确的解析式

进化算法的基本步骤

- 1 (初始化)置进化代数 $t = 0$ ，随机生成 N 个个体作为初始种群 $\bar{X}(t)$;
- 2 (个体评价)按照一定规则计算出种群 $\bar{X}(t)$ 中每一个个体的适应度;
- 3 (种群进化)将选择算子作用于种群 $\bar{X}(t)$ 得到中间种群 $\bar{X}'(t)$; 将繁殖算子作用于种群 $\bar{X}'(t)$ 得到下一代种群 $\bar{X}(t+1)$;
- 4 (终止检验)如果种群 $\bar{X}(t+1)$ 满足终止准则，则输出 $\bar{X}(t+1)$ 中具有最大适应度的个体作为最优解，否则置进化代数 $t = t + 1$ ，转第2步。

主要的进化算法

- 1 遗传算法(Genetic Algorithm)
- 2 微分进化算法(Differential Evolutionary Algorithm)
- 3 模拟退火算法(Simulated Annealing Algorithm)[不属于进化算法]

遗传算法(GA)的主要步骤

- 1 编码，将变量表达成个体(或染色体)的形式；
- 2 初始化种群，确定初始迭代点集合。
- 3 个体评价，赋予较优的个体以较大的适应度。
- 4 选择，让适应度较大的个体有较大的生存机会。
- 5 交叉，让选择后的种群个体间交换遗传信息。
- 6 变异，对个体以一定的概率给予扰动。
- 7 终止检验，终止算法或重新开始进化。

编码

编码的定义

映射 $e : H_L = \{A = c_1c_2 \dots c_L | c_i \in \Gamma\} \rightarrow \mathbb{R}^n$, 满足 $\forall A \in H_L$, 存在唯一的 $X = e^{-1}(A) \in \mathbb{R}^n$ 与之对应, 则称 A 为 X 的编码, X 为 A 的解码。从生物学意义上讲, A 为个体的基因型, X 为个体的表现型。

编码方法:二进制编码

有决策变量 $x \in \mathbb{R}^n$, $x_i \in [a_i, b_i]$, 编码精度为 ϵ , 则编码长度 L_i 应满足

$$\frac{b_i - a_i}{2^{L_i} - 1} < \epsilon$$

总编码长度为 $L = \sum_{i=1}^n L_i$ 。解码公式为

$$(c_1 c_2 \dots c_{L_i}) \rightarrow x_i, \quad x_i = a_i + \frac{b_i - a_i}{2^{L_i} - 1} \sum_{j=1}^{L_i} c_j \times 2^{j-1}$$

编码方法：实数编码和符号编码

实数编码

变量 $X = (x_1, x_2, \dots, x_n)$ 直接编码为个体 $x_1, x_2, \dots, x_n$

实数编码的优点：

- 1 不存在不连续问题
- 2 不需要解码过程
- 3 遗传操作简单，适合于高精度的数值优化

符号编码

利用符号对个体进行编码。适合于非数值优化和组合优化。

实数编码和符号编码都缺乏理论支持。

编码：示例

示例1

若精度要求 $\epsilon \leq 0.01$ ，决策变量位于区间 $[-1, 2]$ 中，按照二进制编码的要求，单个变量的编码长度至少应该满足

$$2^{L_i} - 1 \geq 300; L_i = 9$$

示例2

若决策变量 $x \in \mathbb{R}^2$ ，则个体的实数编码为 x_1x_2 ；
在非数值优化中，个体可以编码为AATCD的形式。

适应度

适应度 $J(A)$ 用于度量个体的生存能力，类似于目标函数的值。对于求最小化 $\min f(x)$ 的问题，常见的适应度函数有

- 1 $F(x) = -f(x)$
- 2 $F(x) = \max(c_{\max} - f(x), 0)$
- 3 $F(x) = 1/(1 + c + f(x))$

为了调节个体适应度的差异，可对适应度函数进行适当的尺度变换，如将适应度都加上一个特别大的数。这样一来，个体适应度的差异将明显减少，有助于增加种群的多样性，但减缓了种群的进化速度。

选择方法

“适者生存”的理念通过一定的选择方法进行实现。

轮盘赌选择方法

按照个体 A_i 的适应度 $J(A_i)$ 占种群总适应度 $\sum_{j=1}^N J(A_j)$ 的比例划分赌盘，转动赌盘 N 次，每次选出赌盘指针所指示的个体。

锦标赛选择方法

每次随机选择 s 个个体，选出其中适应度最大的个体，重复 N 次完成选择操作。 s 称为竞赛规模， $s = 2$ 时为二元锦标赛选择。

交叉或基因重组

基因重组是结合来自父代交配种群中的信息产生新的个体的过程。对二进制编码可以采用单点交叉、多点交叉和均匀交叉等方式；对于实数编码，通常采用算术交叉和启发式交叉等。其中，算术交叉为

$$C(x^{(1)}, x^{(2)}) = \alpha x^{(1)} + (1 - \alpha)x^{(2)}$$

启发式交叉为

$$C(x^{(1)}, x^{(2)}) = \begin{cases} x^{(2)} + \mu(x^{(2)} - x^{(1)}) & \text{if } x^{(2)} \text{ is better than } x^{(1)} \\ x^{(1)} + \mu(x^{(1)} - x^{(2)}) & \text{if } x^{(1)} \text{ is better than } x^{(2)} \end{cases}$$

变异

变异就是个体基因按照变异概率 P_m 扰动产生的变化。对二进制编码可以采取均匀变异，即对个体编码每个基因以一定的概率发生反转，0变为1，1变为0。

对于实数编码，通常采取正态变异，即

$$M(x) = x + N(0, \sigma^2)$$

或采用非均匀变异：

$$M(x) = \begin{cases} x + \Delta(t, b - x) & \text{if } rand(0, 1) = 0 \\ x + \Delta(t, x - a) & \text{otherwise} \end{cases}$$

a 、 b 分别表示 x 的下界和上界。 $rand(0, 1)$ 表示等概率的取值0或1。 $\Delta(t, y)$ 表示 $[0, y]$ 范围内服从非均匀分布的一个随机数。

算法终止准则

终止准则通常包括两种类型：

- 1 达到了预先设定的进化代数
- 2 种群达到了某种平衡状态（如个体差异非常小）

示例

求如下优化问题的全局最优解:

$$\min_{x \in [-2, 2]} e^{-x^2} \sin(6x)$$

按照以上步骤，利用遗传算法求其全局最优解。要求自变量的精度为 $\epsilon \leq 0.1$ 。

示例: Step 1-编码

假定采用二进制编码，由于精度要求为 $\epsilon \leq 0.1$ ，而区间长度为4，可知编码长度 L 应该满足：

$$\frac{4}{2^L - 1} < \epsilon$$

由此可得编码长度 L 至少应该大于等于6。

示例: Step2-初始化

初始化种群，确定迭代初始点集合。假定种群规模 $N = 6$ ，初始种群一般是随机生成的，不妨设初始种群

$$\bar{X}^{(0)} = \{100001, 010010, 001010, 100010, 010001, 110010\}$$

示例: Step3-适应度计算

计算每个个体的适应度，该问题是一个求极小化的问题，因此，设适应度函数为

$$F(x) = -e^{-x^2} \sin(6x)$$

对种群进行解码，可得

$$\bar{x}^{(0)} = \{0.0952, -0.8571, -1.3651, 0.1587, -0.9206, 1.1746\}$$

相应的适应度为

$$F(\bar{x}^{(0)}) = \{-0.5358, -0.4360, 0.1464, -0.7944, -0.2951, -0.1742\}$$

示例: Step4-选择

选择就是让适应度较大的个体有较大的生存机会，一般采用比例选择方式，即每个个体在一次选择中被选中的机会 P_r 与适应度成正比，假定经过 6 次选择之后，得到的种群为

$$\bar{X}'^{(0)} = \{100001, 001010, 001010, 010001, 010001, 110010\}$$

示例: Step5-交叉

交叉就是让选择后的种群个体间交换遗传信息，设交叉概率 $P_c = 0.6$ ；采用单点交叉方式，在母体选择中有三个个体被选中100001, 001010, 110010，从中选择一对进行交叉，随机选择交叉点为2，如

$$\left(\begin{array}{cc|cccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{array} \right)$$

交叉后得到两个新的个体110001和100010，用这两个新个体替换原来的个体，得到新的种群

$$\bar{X}''^{(0)} = \{110001, 001010, 001010, 010001, 010001, 100010\}$$

示例: Step6-变异

假定个体发生均匀变异，即每个基因座发生变异的概率相等，令变异概率 $P_m = 0.02$ 。若在第3个个体的第4个基因座发生变异 $0 \rightarrow 1$ ，经过变异后得到新一代种群。

$$\bar{X}^{(1)} = \{110001, 001010, 001110, 010001, 010001, 100010\}$$

检验这组新的种群是否满足停止规则，如是则停止算法；否则继续迭代。

概述

Matlab中有一个全局优化工具箱(Global Optimization Toolbox), 内置了遗传算法GA。所针对的优化问题模型与优化工具箱中模型类似, 能够对有约束的数值优化问题进行求解。所采用的函数为ga。

函数ga的调用方式

```
x = ga(fitnessfcn,nvars)
x = ga(fitnessfcn,nvars,A,b)
x = ga(fitnessfcn,nvars,A,b,Aeq,beq)
x = ga(fitnessfcn,nvars,A,b,Aeq,beq, LB, UB)
x = ga(fitnessfcn,nvars,A,b,Aeq,beq, LB, UB, ...
       nonlcon)
x = ga(fitnessfcn,nvars,A,b,Aeq,beq, LB, UB, ...
       nonlcon,options)
```

fitnessfcn表示适应度函数，nvars表示决策变量的维数。其他参数与fmincon的参数相同。

函数ga的输出

```
[x,fval] = ga(...)  
[x,fval,exitflag] = ga(...)  
[x,fval,exitflag,output] = ga(...)  
[x,fval,exitflag,output,population] = ga(...)  
[x,fval,exitflag,output,population,scores] = ga(...)
```

population表示最终的种群，scores表示最终种群的评价分值。

示例

$$f(x) = 100(x_1^2 - x_2)^2 + (1 - x_1)^2 \rightarrow \min$$

$$x_1x_2 + x_1 - x_2 + 1.5 \leq 0$$

$$10 - x_1x_2 \leq 0$$

$$0 \leq x_1 \leq 1$$

$$0 \leq x_2 \leq 13$$

示例

%适应度函数

```
function y = simple_fitness(x)
```

```
y = 100*(x(1)^2 - x(2))^2 + (1 - x(1))^2;
```

%约束

```
function [c, ceq] = simple_constraint(x)
```

```
c = [1.5 + x(1)*x(2) + x(1) - x(2); ...
```

```
-x(1)*x(2) + 10];
```

```
ceq = [];
```

示例

按照如下格式调用ga

```
ObjectiveFunction = @simple_fitness;  
nvars = 2;        % Number of variables  
LB = [0 0];       % Lower bound  
UB = [1 13];      % Upper bound  
ConstraintFunction = @simple_constraint;  
[x,fval] = ga(ObjectiveFunction,nvars,...  
              [],[],[],[],LB,UB,ConstraintFunction)
```

输出结果为

```
x = 0.8122    12.3122  
fval = 1.3578e+004
```


概述

DE算法是由Storn和Price于1995年提出的一种简单有效的进化算法。1996年，该算法参加了首届IEEE进化算法大赛，被证明为是所有参赛算法中最快的进化算法。对于连续变量的函数优化问题，DE算法能够更快更稳定的收敛到全局最优解。

基本思想

对种群中的每个个体 i ，从当前种群中随机选择3个点，以其中一个点为基础，另外两个点为参照进行扰动，所得到的点与个体 i 交叉后进行自然选择，保留较优者，实现种群的进化。

初始化

Step 1: 初始化

输入进化参数：种群规模 N ，交叉概率 P_c ，交叉因子 $F \in (0, 1)$ ，进化代数 $t = 0$ ，决策变量的上下界 $[lb, ub]$ ；随机生成初始种群 $\bar{x}(0) = \{x^{(1)}(0), x^{(2)}(0), \dots, x^{(N)}(0)\}$ ，其中

$$x^{(i)}(0) = (x_1^{(i)}(0), x_2^{(i)}(0), \dots, x_n^{(i)}(0))^T$$

Remarks: 种群规模 N 通常取 $5n$ 到 $10n$ 之间， n 为变量的维数；交叉概率 P_c 通常取0.1；交叉因子 F 通常取0.5。

个体评价和繁殖

Step 2: 个体评价

计算每个个体 $x^{(i)}(t)$ 的目标函数值 $f(x^{(i)}(t))$ 。

Step 3: 繁殖

对种群中每个个体 $x^{(i)}(t)$ ，随机生成三个互不相同的随机整数 $r_1, r_2, r_3 \in [1, N]$ 和一个随机整数 $j_r \in [1, n]$

$$x_j^{(i)'}(t) = \begin{cases} x_j^{(r_1)}(t) + F[x_j^{(r_2)}(t) - x_j^{(r_3)}(t)] & \text{if } K < P_c \text{ or } j = j_r \\ x_j^{(i)}(t) & \text{otherwise} \end{cases}$$

其中， $K = rand[0, 1]$ 。

产生新种群

Step 4: 选择

选择新的个体:

$$x_j^{(i)}(t+1) = \begin{cases} x_j^{(i)'}(t) & \text{if } f(x_j^{(i)'}(t)) < f(x_j^{(i)}(t)) \\ x_j^{(i)}(t) & \text{otherwise} \end{cases}$$

Step 5: 终止检验

如果新种群 $x(t+1)$ 满足终止规则, 则输出种群中目标函数值最小的个体作为最优解, 否则转Step 2。

微分进化算法的优缺点

优点

- 在求解非凸、多峰、非线性函数的优化问题时表现出极强的稳定性
- 精度要求相同时，DE算法的收敛速度更快
- 特别擅长求解多变量的函数优化问题
- 操作简单，易于编程实现

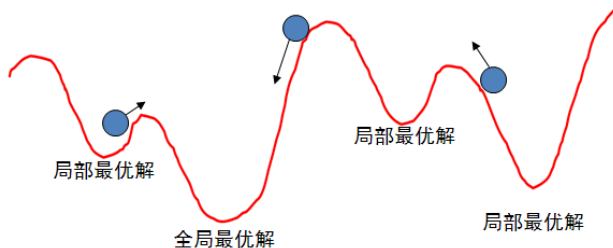
缺点

算法的关键步骤变异操作是基于群体的差异向量信息来修正各个体的值, 随着进化代数的增加, 各个体之间的差异化信息在逐渐缩小, 以至于后期收敛速度变慢, 甚至有时会陷入局部最优解。

概述

模拟退火(simulated annealing, SA)是一种通用的随机搜索算法, 1953年由Metropolis等人提出。其基本思想是把某类优化问题的求解过程与统计热力学的热平衡问题进行对比, 试图通过模拟高温物体退火的过程来找到优化问题的全局最优解或近似全局最优解。模拟退火算法来源于金属退火原理, 将金属加温至充分高, 再让其徐徐冷却, 加温时, 金属内部粒子随温升变为无序状, 内能增大, 而徐徐冷却时粒子渐趋有序, 在每个温度都达到平衡态, 最后在常温时达到基态, 内能减为最小。

基本思想



SA方案就是类似于沿水平方向给圆球一个作用力，若该力足够大且小球所处的低谷并不很深，小球受水平力作用会从该低谷滚出，落入另一个低谷，然后受水平力作用又滚出，如此不断滚动。如果水平力掌握适当，小球很有可能停留在最深的低谷中，这个最深的低谷就是(近似)全局最优解。

算法步骤

模拟退火算法利用模拟热力学中经典粒子系统的降温过程来求解规划问题的极值。基本步骤如下：

- 1 令 $k = 0$ ；给定初始温度 t_k 以及初始点 $x^{(k)}$ ，计算该点的目标函数值 $f(x^{(k)})$ ；
- 2 随机产生扰动 Δx ，得到新点 $x = x^{(k)} + \Delta x$ ，计算新点的目标函数值 $f(x)$ 及两个函数值的差 $\Delta f = f(x) - f(x^{(k)})$ ；
- 3 若 $\Delta f < 0$ ，则接受新点 x 作为下一次模拟退火的起始点，即 $x^{(k+1)} = x$ ；
- 4 若 $\Delta f \geq 0$ ，则计算新点的接受概率 $p = \exp(-\Delta f/t_k)$ ，以概率 p 接受 x 作为下一次退火的起始点；
- 5 令 $k = k + 1$ ，返回第2步。

算法：参数选择

模拟退火的执行过程中，算法效果取决于一组控制参数的选择。模拟退火的关键参数包括

- 1 初始温度
- 2 温度下降方法
- 3 每一温度迭代长度
- 4 终止准则

算法：初始温度的控制

温度的初始值设置是影响模拟退火算法全局搜索性能的重要因素之一。初始温度高，则搜索到全局最优解的可能性大，但因此要花费大量的计算时间；反之，则可节约计算时间，但全局搜索性能可能受到影响，使搜索到全局最优解的可能性减小。实际应用过程中，初始温度一般需要依据实验结果进行若干次调整，一般温度初值 t_0 取较大值较为合理。

算法：温度下降方法

温度下降方法的确定是模拟退火中影响全局搜索性能的重要因素。如果温度下降过快可能就会丢失极值点；如果温度下降过慢，收敛速度又会大大降低，导致计算时间过长。在实际问题中，通常采用两种直观的温度下降方法：

- 1 每一步温度以相同比率下降，即

$$t_{k+1} = \partial t_k, k \geq 0, 0 < \partial < 1, \partial \text{为降温系数}$$

- 2 每一步温度以相同迭代长度下降，即

$$t_k = t_0(N-k)/N, t_0 \text{为初始温度}, N \text{为温度下降总次数}$$

算法：每一温度迭代长度的控制

模拟退火的全局搜索与每一温度的迭代长度密切相关。一般的，同一温度下的充分搜索是必要的，但需以计算时间增加为代价。实际中常根据问题的特点来设置合理的迭代长度，常用两种方法：

- ① 固定的迭代步数，即在每一温度都设置相同的迭代步数；
- ② 高温时，各状态被接受的概率基本相同，且几乎都被接受，可使同一温度的迭代步数尽量少；温度逐渐变低后，越来越多的状态被拒绝，则可相应增加迭代的步数。可给定一个迭代步数上限 S 和接受次数上限 R ，当某一温度的实际接受次数等于 R 时，不再迭代，否则迭代到上限步数 S 。

算法：终止准则

模拟退火的终止准则比较直观：

- 1 零度法：给定一个较小的正数 n ，当温度低于该值时，迭代停止；
- 2 循环总数控制法：设定温度下降次数 N ，温度迭代次数达到该值时，迭代停止；
- 3 基于不改进规则的控制法：两次相邻的迭代中局部最优解没有改进，迭代停止；
- 4 接受概率终止准则：给定一个较小的概率 P ，在一个温度和给定的迭代步数内，除当前解之外，其他状态的接受概率都小于 P ，迭代终止。

Matlab中的模拟退火算法

在Matlab全局优化工具箱中，模拟退火方法只用于求解无约束或区间约束的非线性规划问题。

$$\begin{aligned} f(x) &\rightarrow \min \\ lb &\leq x \leq ub \end{aligned}$$

Matlab中定义的模拟退火函数为simulannealbnd。

函数simulannealbnd详解

调用方式

```
[x fval] = simulannealbnd(@objfun, x0, ...  
                           lb, ub, options)
```

@objfun表示目标函数的函数句柄；

x0表示初始值；

lb和ub分别表示决策变量的下界和上界；

options表示优化选项。

示例

$$f(x) = (4 - 2.1x_1^2 + \frac{1}{3}x_1^4)x_1^2 + x_1x_2 + (-4 + 4x_2^2)x_2^2 \rightarrow \min$$

试利用模拟退火方法求解该无约束规划。

示例

定义目标函数

```
function y = simple_objective(x)
    y = (4 - 2.1*x(1)^2 + x(1)^4/3)*x(1)^2 + ...
        x(1)*x(2) + (-4 + 4*x(2)^2)*x(2)^2;
```

求解

```
ObjectiveFunction = @simple_objective;
X0 = [0.5 0.5]; % Starting point
[x,fval,exitFlag,output] = simulannealbnd(...
    ObjectiveFunction,X0)
```

计算结果为 $x = (-0.0891, 0.7122)^T$ 。

总结

- 1 全局优化问题的基本概念
- 2 全局优化问题的求解思路
- 3 全局优化问题的进化算法
 - ▶ 遗传算法
 - ▶ 微分进化算法
 - ▶ 模拟退火算法[严格的说，不属于进化算法]
- 4 利用Matlab求解全局优化问题